

Homework 05

Neel (nnigam2)

All work is in Python (NumPy, pandas, matplotlib, scikit-learn). Each simulation question sets its own seed with `numpy.random.default_rng`: Question 1 uses seed 432, Question 2 uses 4322, and Question 3 uses 43203. KNN uses equal neighbor weights throughout, and no intercept column is added to any KNN predictor matrix.

```
import math
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
from IPython.display import display
from sklearn.neighbors import KNeighborsRegressor
from sklearn.linear_model import Lasso
from sklearn.model_selection import KFold

pd.set_option("display.precision", 5)

def f_true(X):
    """f(x) = 0.5 x1 + sin(x2) - 0.3 x3^2 (uses only the first three columns)."""
    return 0.5 * X[:, 0] + np.sin(X[:, 1]) - 0.3 * X[:, 2] ** 2

def knn_predict_all_k(X_train, y_train, X_query, ks):
    """KNN regression predictions for several k at once (Euclidean, equal weights).
    Returns an array of shape (n_query, len(ks))."""
    nn = KNeighborsRegressor(n_neighbors=max(ks),
                             algorithm="brute").fit(X_train, y_train)
    idx = nn.kneighbors(X_query, return_distance=False) # nearest first
    cum = np.cumsum(y_train[idx], axis=1) # running sums of
    neighbor responses
    return np.column_stack([cum[:, k - 1] / k for k in ks])
```

Question 1: Estimate the bias and variance of KNN

(a) True mean and one KNN fit

```
SEED1 = 432
rng = np.random.default_rng(SEED1)

n, sigma = 400, 0.1
x0 = np.array([[0.5, 0.7, 1.0]])
ks1 = list(range(1, 30, 2))
f0 = f_true(x0)[0]

X = rng.standard_normal((n, 3))
y = f_true(X) + sigma * rng.standard_normal(n)
pred_one = knn_predict_all_k(X, y, x0, ks1)[0]
pd.DataFrame({"k": ks1, "prediction": pred_one, "prediction - f(x0)": pred_one - f0}).round(4)
```

	k	prediction	prediction - f(x0)
0	1	0.6657	0.0715
1	3	0.6207	0.0265
2	5	0.5167	-0.0775
3	7	0.4747	-0.1195
4	9	0.4140	-0.1803
5	11	0.4020	-0.1922
6	13	0.4127	-0.1815
7	15	0.3552	-0.2390
8	17	0.3727	-0.2215
9	19	0.3775	-0.2167
10	21	0.4157	-0.1785
11	23	0.4387	-0.1555
12	25	0.4595	-0.1347
13	27	0.4268	-0.1674
14	29	0.4419	-0.1523

The true mean response is

$$f(\mathbf{x}_0) = 0.5(0.5) + \sin(0.7) - 0.3(1)^2 = 0.25 + 0.64422 - 0.3,$$

so $f(\mathbf{x}_0) = 0.59422$.

The table gives the KNN prediction at $\mathbf{x}_0$ for each k from one training dataset (seed 432). With a single dataset the errors jump around from one k to the next, so the repeated simulation below is what shows the systematic pattern.

(b) Bias, variance, and MSE over 200 repetitions

```
R = 200
P = np.empty((R, len(ks1)))          # 200 x 15 prediction matrix
for r in range(R):
    X = rng.standard_normal((n, 3))
    y = f_true(X) + sigma * rng.standard_normal(n)
    P[r] = knn_predict_all_k(X, y, x0, ks1)[0]

mean_pred = P.mean(axis=0)
bias = mean_pred - f0
var = ((P - mean_pred) ** 2).mean(axis=0)      # divisor 200
mse = ((P - f0) ** 2).mean(axis=0)
gap = mse - (bias ** 2 + var)

q1 = pd.DataFrame({"k": ks1, "bias": bias, "squared bias": bias ** 2, "variance":
var,
                    "MSE": mse, "squared bias + variance": bias ** 2 + var,
                    "MSE - (bias2 + var)": gap})
k_best1 = ks1[int(np.argmin(mse))]
k_cross = ks1[int(np.argmax(bias ** 2 > var))]
print(f"Prediction matrix shape: {P.shape}")
print(f"Max |MSE - (squared bias + variance)| = {np.max(np.abs(gap)):.2e}")
q1.style.format({"k": "{:d}", "MSE - (bias2 + var)": "{:.1e}"}, precision=5)
```

```
Prediction matrix shape: (200, 15)
Max |MSE - (squared bias + variance)| = 5.20e-17
```

	k	bias	squared bias	variance	MSE	squared bias + variance	MSE - (bias ² + var)
0	1	-0.04456	0.00199	0.03512	0.03710	0.03710	6.9e-18
1	3	-0.03681	0.00135	0.01943	0.02078	0.02078	1.0e-17
2	5	-0.04424	0.00196	0.01502	0.01698	0.01698	0.0e+00
3	7	-0.05366	0.00288	0.01198	0.01486	0.01486	-1.0e-17
4	9	-0.06209	0.00386	0.01088	0.01474	0.01474	2.3e-17

	k	bias	squared bias	variance	MSE	squared bias + variance	MSE - (bias ² + var)
5	11	-0.06452	0.00416	0.00979	0.01395	0.01395	5.2e-18
6	13	-0.06718	0.00451	0.00981	0.01432	0.01432	-2.8e-17
7	15	-0.07175	0.00515	0.00876	0.01390	0.01390	-2.3e-17
8	17	-0.07738	0.00599	0.00807	0.01406	0.01406	-1.6e-17
9	19	-0.08589	0.00738	0.00768	0.01506	0.01506	-2.3e-17
10	21	-0.09007	0.00811	0.00779	0.01590	0.01590	2.1e-17
11	23	-0.09738	0.00948	0.00709	0.01657	0.01657	1.0e-17
12	25	-0.10415	0.01085	0.00719	0.01804	0.01804	1.0e-17
13	27	-0.11070	0.01226	0.00629	0.01854	0.01854	5.2e-17
14	29	-0.11574	0.01340	0.00622	0.01961	0.01961	4.5e-17

```
plt.figure()
plt.plot(ks1, bias ** 2, marker="o", label="Squared bias")
plt.plot(ks1, var, marker="o", label="Variance")
plt.plot(ks1, mse, marker="o", label="MSE")
plt.xlabel("k")
plt.ylabel("Value")
plt.title("KNN at x0: squared bias, variance, MSE")
plt.legend()
plt.show()
```

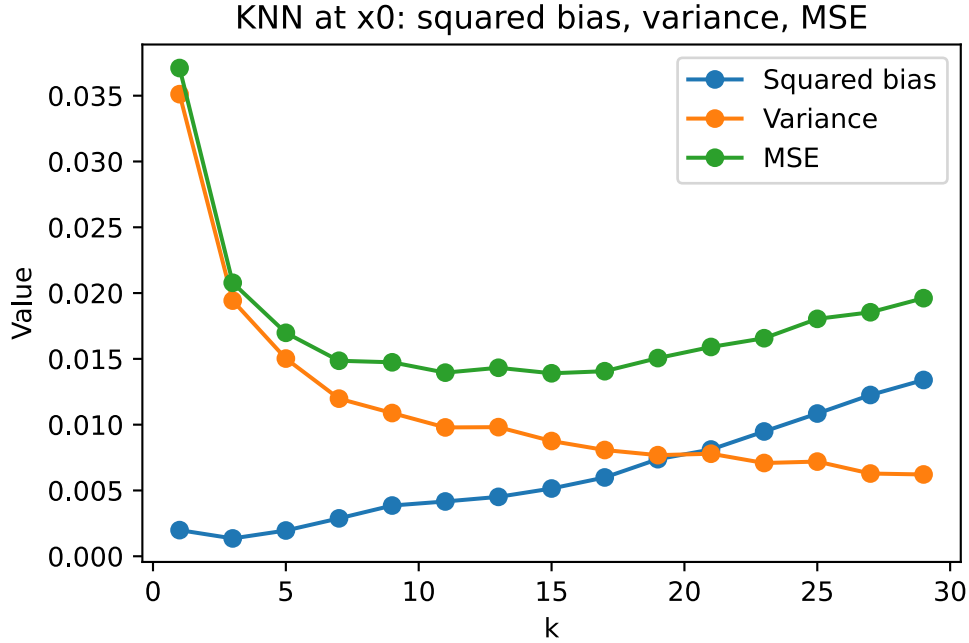


Figure 1: Squared bias, variance, and MSE of the KNN estimate of $f(x_0)$ over 200 training sets (original plot).

Verification. The largest absolute gap between the MSE and squared bias plus variance is $5.2e-17$, which is floating-point rounding. The identity holds exactly with divisor 200. Write $\bar{m}$ for the mean of the 200 predictions. The cross term in $\frac{1}{R} \sum_r (\hat{f}_r - \bar{m} + \bar{m} - f_0)^2$ is $2(\bar{m} - f_0) \cdot \frac{1}{R} \sum_r (\hat{f}_r - \bar{m})$, and that sum is exactly zero.

Pattern.

- **Variance falls steadily with k ,** from 0.0351 at $k = 1$ to 0.0062 at $k = 29$. Averaging more responses reduces the noise part roughly like σ^2/k . It also smooths out the randomness in where the nearest training points fall. That second source is why the variance at $k = 1$ is well above $\sigma^2 = 0.01$: the single nearest neighbor sits at a different location in each training set.
- **Squared bias is tiny for small k and rises steadily after that,** from about 0.0018 for $k \leq 5$ to 0.0134 at $k = 29$. The small wiggle among $k = 1, 3, 5$ is Monte Carlo noise from using 200 repetitions. The bias is negative for every k . Larger neighborhoods reach farther from $\mathbf{x}_0$, into a region where f is curved (concave in x_2 and x_3). The neighbors are also not spread symmetrically around $\mathbf{x}_0$, because the design density is higher toward the origin. So the local average drifts below $f(\mathbf{x}_0)$.
- **The curves cross and the MSE is U-shaped.** Squared bias first exceeds variance at $k = 21$. MSE falls quickly at first, reaches its minimum 0.0139 at $k = 15$, and is nearly flat for k between about 11 and 17. Past that range, bias dominates and MSE rises again.

Comparison with ridge and lasso. The penalty λ plays the same role in ridge and lasso that k plays here. Both control model complexity. A small k is like a small λ : a flexible fit with low

bias and high variance. A large k is like a large λ : heavy smoothing or shrinkage, with high bias and low variance. KNN's effective degrees of freedom are about n/k , just as the ridge degrees of freedom $\text{tr}(\mathbf{H}_\lambda)$ shrink as λ grows. In both cases the MSE-optimal tuning value sits in the middle.

There are two differences:

- **Where the bias comes from.** Ridge and lasso shrink a global linear model toward zero, and if the true f is nonlinear, some bias remains even at $\lambda = 0$. KNN's bias comes from averaging over a local neighborhood. It goes to zero as the neighborhood shrinks, at the cost of variance.
- **How finely the dial turns.** k is discrete, whereas λ varies continuously.

Question 2: Compare KNN with lasso

```
SEED2 = 4322
rng = np.random.default_rng(SEED2)
p, n_tr, n_te = 30, 400, 1000
lambdas = np.logspace(-4, 0, 41)

def lasso_fit(X, y, lam):
    """Week 4 objective:  $(1/2n)||y - X\beta||^2 + \lambda ||\beta||_1$  on centered and
    standardized X (divisor = sample size) and centered y. The intercept is fitted
    separately and left unpenalized. Returns a predictor that applies the fitted
    transformations unchanged, plus the slopes."""
    mu, sd, ybar = X.mean(axis=0), X.std(axis=0, ddof=0), y.mean()
    Xs = (X - mu) / sd
    beta = Lasso(alpha=lam, fit_intercept=False, max_iter=200000,
tol=1e-10).fit(Xs, y - ybar).coef_
    return (lambda Xnew: ybar + ((Xnew - mu) / sd) @ beta), beta

def lasso_cv(X, y, lambdas, seed):
    """Ten-fold CV. Means and scales are re-estimated on each fold's training
    portion."""
    kf = KFold(n_splits=10, shuffle=True, random_state=seed)
    cv_err = np.zeros(len(lambdas))
    for tr, va in kf.split(X):
        for j, lam in enumerate(lambdas):
            pred, _ = lasso_fit(X[tr], y[tr], lam)
            cv_err[j] += np.mean((y[va] - pred(X[va])) ** 2)
    cv_err /= 10
    return lambdas[int(np.argmin(cv_err))], cv_err

def gen_setting(n, setting, rng):
```

```

if setting == 1:
    X = rng.standard_normal((n, p))
else:
    Sigma = np.full((p, p), 0.8)
    np.fill_diagonal(Sigma, 1.0)
    X = rng.multivariate_normal(np.zeros(p), Sigma, size=n)
return X, f_true(X) + sigma * rng.standard_normal(n)

```

(a) Results

```

rows, diag = [], {}
for s in (1, 2):
    Xtr, ytr = gen_setting(n_tr, s, rng)
    Xte, yte = gen_setting(n_te, s, rng)

    knn = KNeighborsRegressor(n_neighbors=5, algorithm="brute").fit(Xtr, ytr)
    mse_knn = np.mean((yte - knn.predict(Xte)) ** 2)

    lam, cv_err = lasso_cv(Xtr, ytr, lambdas, seed=SEED2 + s)
    pred, beta = lasso_fit(Xtr, ytr, lam)
    mse_lasso = np.mean((yte - pred(Xte)) ** 2)
    nz = np.flatnonzero(beta != 0)

    # neighborhood diagnostics in the three relevant coordinates
    dist, idx = knn.kneighbors(Xte)
    rel_nn = np.linalg.norm(Xtr[idx][:, :, :3] - Xte[:, None, :3], axis=2).mean()
    # separate generator so this diagnostic does not shift the simulation's
    random stream
    rand_rows = np.random.default_rng(0).integers(0, n_tr, n_te)
    rel_rand = np.linalg.norm(Xtr[rand_rows, :3] - Xte[:, :3], axis=1).mean()

    rows.append({"Setting": s, "5NN test MSE": mse_knn, "Lasso test MSE":
mse_lasso,
                "Selected  $\lambda$ ": lam, "Nonzero lasso slopes": len(nz)})
    diag[s] = dict(var_y=yte.var(), cv=cv_err.min(), beta=beta[:3],
n_noise=int(np.sum(nz >= 3)),
                noise_ll=np.abs(beta[3:]).sum(), d5=dist[:, -1].mean(),
                rel_nn=rel_nn, rel_rand=rel_rand)

q2 = pd.DataFrame(rows)
display(q2.style.format({"Selected  $\lambda$ ":
"{:.4g}"}, precision=4).hide(axis="index"))

```

```
pd.DataFrame({
    "Var(y) in test set": [diag[s]["var_y"] for s in (1, 2)],
    "Lasso min CV MSE": [diag[s]["cv"] for s in (1, 2)],
    "Lasso slope X1": [diag[s]["beta"][0] for s in (1, 2)],
    "Lasso slope X2": [diag[s]["beta"][1] for s in (1, 2)],
    "Lasso slope X3": [diag[s]["beta"][2] for s in (1, 2)],
    "Nonzero noise slopes": [diag[s]["n_noise"] for s in (1, 2)],
    " $\Sigma$ |slope| on X4–X30": [diag[s]["noise_ll"] for s in (1, 2)],
    "Mean dist. to 5th NN (30-d)": [diag[s]["d5"] for s in (1, 2)],
    "Mean (X1,X2,X3) dist. to the 5 NNs": [diag[s]["rel_nn"] for s in (1, 2)],
    "Mean (X1,X2,X3) dist. to a random training point": [diag[s]["rel_rand"] for
s in (1, 2)],
}, index=["Setting 1", "Setting 2"]).T.round(3)
```

Setting	5NN test MSE	Lasso test MSE	Selected λ	Nonzero lasso slopes
1	0.6229	0.2752	0.02512	10
2	0.1802	0.3114	0.01995	10

	Setting 1	Setting 2
Var(y) in test set	0.853	1.407
Lasso min CV MSE	0.211	0.203
Lasso slope X1	0.459	0.360
Lasso slope X2	0.613	0.530
Lasso slope X3	0.038	0.136
Nonzero noise slopes	7.000	7.000
Σ slope on X4–X30	0.160	0.142
Mean dist. to 5th NN (30-d)	5.748	2.963
Mean (X1,X2,X3) dist. to the 5 NNs	1.636	0.824
Mean (X1,X2,X3) dist. to a random training point	2.243	2.094

```
d1, d2 = diag[1], diag[2]
knn1, knn2 = q2.loc[0, "5NN test MSE"], q2.loc[1, "5NN test MSE"]
las1, las2 = q2.loc[0, "Lasso test MSE"], q2.loc[1, "Lasso test MSE"]
# population MSE floor for ANY linear predictor (derived in part c)
lin_floor = (1 - math.exp(-2)) / 2 - math.exp(-1) + 0.09 * 2 + sigma ** 2
```

The first table is the required summary (seed 4322; the same training and test data are used for both methods within a setting). The second table gives diagnostics used in the discussion below.

The noise variance is 0.01, and the variance of the test responses (0.853 and 1.407) is the MSE of simply predicting the mean, which sets the scale.

(b) Comparing the two settings

In **Setting 1**, lasso wins clearly: 0.275 against 0.623 for 5NN. In **Setting 2**, the ranking flips. 5NN improves dramatically to 0.180, while lasso's test MSE is 0.311. Lasso's population error floor is the same in both settings (see part c), so its change mainly reflects the single data draw.

Why correlation helps the Euclidean neighborhoods here. In Setting 1 all 30 coordinates are independent, and 27 of them are irrelevant. Each coordinate contributes equally to the squared Euclidean distance, so the distance is dominated by noise coordinates. The five “nearest” neighbors are at an average distance of 5.75 in 30 dimensions. Measured only on (X_1, X_2, X_3) , they are at average distance 1.64, not much closer than a randomly chosen training point (2.24). So the neighbors carry little information about $f(\mathbf{x}_0)$.

In Setting 2, $\Sigma = 0.2\mathbf{I} + 0.8\mathbf{1}\mathbf{1}^T$, so each $X_j = \sqrt{0.8}W + \sqrt{0.2}E_j$ for a common factor W . The covariance has one eigenvalue $0.2 + 0.8(30) = 24.2$ and 29 eigenvalues equal to 0.2. About 81% of the total variance lies along a single direction, so the data are effectively low-dimensional. Every coordinate, including the “irrelevant” ones, now carries information about W , which is also 80% of the variance of X_1, X_2 , and X_3 . Matching on the full 30-dimensional distance mostly matches on W , and therefore on the relevant variables. The neighbors are now at average distance 0.82 in (X_1, X_2, X_3) , compared with 2.09 for a random training point. That is much more useful local information.

Why lasso does not use the covariates the way KNN does. Lasso estimates a separate slope for each covariate, and the ℓ_1 penalty shrinks the slopes of unhelpful covariates to exactly zero. Irrelevant covariates are therefore largely removed from the prediction. Only 7 and 7 noise covariates survived, with total absolute slope 0.16 and 0.14, compared with slopes of roughly 0.35–0.6 on X_1 and X_2 .

KNN has no such mechanism. Its distance weights all 30 coordinates equally regardless of their relevance, so irrelevant coordinates dilute the neighborhoods. This is why irrelevant covariates hurt KNN far more than lasso in Setting 1. Correlation does make lasso's job slightly harder: the effect of X_1 can be partly credited to correlated covariates. The X_1 slope was 0.46 in Setting 1 and 0.36 in Setting 2.

(c) Nonlinearity and general claims

The lasso fit can only produce a linear function of the covariates, so part of f is out of its reach:

- $\sin(X_2)$. The best linear stand-in has slope $E[X \sin X] = e^{-1/2} \approx 0.607$. That is essentially the fitted X_2 slope: 0.613 and 0.530.
- $-0.3X_3^2$. This term is symmetric in X_3 , so it has no linear component at all. The fitted X_3 slopes are near zero: 0.038 and 0.136.

These two gaps put a floor under the error of any linear predictor. For jointly Gaussian covariates, Stein's identity shows that both leftovers, $\sin X_2 - e^{-1/2}X_2$ and $X_3^2 - 1$, are uncorrelated with

every linear combination of the covariates, in both settings. So no linear predictor can do better than

$$\text{Var}(\sin X_2) - e^{-1} + 0.09 \text{Var}(X_3^2) + \sigma^2 = \frac{1 - e^{-2}}{2} - e^{-1} + 0.18 + 0.01,$$

which is approximately 0.254.

Lasso's test MSE in Setting 1 (0.275) is close to this floor. Almost all of lasso's error is approximation bias from the linear model, not estimation variance, and no choice of λ can remove it. In Setting 2, the cross-validated MSE (0.203) and the test MSE (0.311) differ noticeably even though the floor is the same. Because X_3^2 is heavy-tailed, a single test set of 1,000 points can land noticeably above or below the population value.

KNN is nonparametric and can in principle capture both nonlinear terms. Its problem is finding good neighbors in 30 dimensions. The comparison therefore mixes two separate effects: lasso's **misspecification bias**, which is the same in both settings, and KNN's **neighborhood quality**, which depends heavily on the covariate structure. If f were linear, lasso would get close to the noise level 0.01 in both settings and would win both times. If the lasso design included X_3^2 and a better term for X_2 , it would likely win as well.

These results do not justify either general claim.

- “Correlation always helps KNN.” Correlation helped here because this particular structure, equicorrelation, creates one dominant direction that is shared with the relevant variables. That lowers the effective dimension in a way that happens to align with f . If the response depended on contrasts such as $X_1 - X_2$, which have small variance (0.4) under this Σ , the Euclidean distance would be dominated by W and would largely ignore the relevant direction. Correlation could then hurt. Correlation confined to irrelevant covariates would not help either.
- “One method is always better.” The winner changed between the two settings. The outcome depends on the true f , n , p , the noise level, and the covariate structure. We also generated only one dataset per setting, and the difference between the Setting 2 cross-validation and test errors shows that single-draw results carry noticeable variability. Settling the ranking would require repeated simulations over a range of settings.

Question 3: Observed dimension and latent dimension

```
SEED3 = 43203
rng = np.random.default_rng(SEED3)
p3, n3, reps3 = 100, 200, 50
ks3 = list(range(2, 83, 8)) # 2, 10, ..., 82
ms = (3, 30)

def gen_latent(n, A, rng):
    Z = rng.standard_normal((n, A.shape[1]))
```

```

X = Z @ A.T + rng.standard_normal((n, p3)) # observed covariates only
y = Z[:, 0] + Z[:, 1] + rng.standard_normal(n)
return X, y

mse3 = {m: np.zeros((reps3, len(ks3))) for m in ms}
pc_share = {m: [] for m in ms}
for r in range(reps3): # repetition 1 is the single experiment of
    part (a)
    for m in ms:
        A = rng.uniform(-2, 2, size=(p3, m)) # new A in every repetition
        Xtr, ytr = gen_latent(n3, A, rng)
        Xte, yte = gen_latent(n3, A, rng)
        preds = knn_predict_all_k(Xtr, ytr, Xte, ks3)
        mse3[m][r] = ((preds - yte[:, None]) ** 2).mean(axis=0)
        ev = np.linalg.eigvalsh(np.cov(Xtr, rowvar=False))[:-1]
        pc_share[m].append(ev[:m].sum() / ev.sum())

avg3 = {m: mse3[m].mean(axis=0) for m in ms}
best3 = {m: ks3[int(np.argmin(avg3[m]))] for m in ms}
share = {m: float(np.mean(pc_share[m])) for m in ms}

```

(a) One dataset for each m

```

pd.DataFrame({"k": ks3, "test MSE (m = 3)": mse3[3][0], "test MSE (m = 30)":
mse3[30][0]}).round(4)

```

	k	test MSE (m = 3)	test MSE (m = 30)
0	2	1.6460	3.1778
1	10	1.4077	2.1920
2	18	1.5230	2.2435
3	26	1.6703	2.2425
4	34	1.7733	2.3062
5	42	1.8768	2.3268
6	50	1.9305	2.3083
7	58	2.0063	2.3189
8	66	2.0785	2.3222
9	74	2.1527	2.3510

	k	test MSE (m = 3)	test MSE (m = 30)
10	82	2.2275	2.3671

This table is the first repetition (seed 43203): one dataset per value of m , with the same training and test observations used for every k .

(b) Average test MSE over 50 repetitions

```
pd.DataFrame({"k": ks3,
              "avg test MSE (m = 3)": avg3[3], "SE (m = 3)": mse3[3].std(axis=0,
ddof=1) / np.sqrt(reps3),
              "avg test MSE (m = 30)": avg3[30], "SE (m = 30)": mse3[30].std(axis=0,
ddof=1) / np.sqrt(reps3)}).round(4)
```

	k	avg test MSE (m = 3)	SE (m = 3)	avg test MSE (m = 30)	SE (m = 30)
0	2	1.6706	0.0240	3.0766	0.0416
1	10	1.2948	0.0189	2.3362	0.0290
2	18	1.3236	0.0198	2.3383	0.0277
3	26	1.3731	0.0218	2.3610	0.0287
4	34	1.4282	0.0229	2.3955	0.0286
5	42	1.4921	0.0251	2.4288	0.0297
6	50	1.5479	0.0260	2.4590	0.0304
7	58	1.6085	0.0269	2.4908	0.0305
8	66	1.6667	0.0277	2.5227	0.0308
9	74	1.7281	0.0285	2.5500	0.0307
10	82	1.7873	0.0293	2.5786	0.0314

```
plt.figure()
for m in ms:
    plt.plot(ks3, avg3[m], marker="o", label=f"m = {m}")
plt.xlabel("k")
plt.ylabel("Average test MSE")
plt.title("KNN with 100 observed covariates: average test MSE (50 reps)")
plt.legend()
plt.show()
```

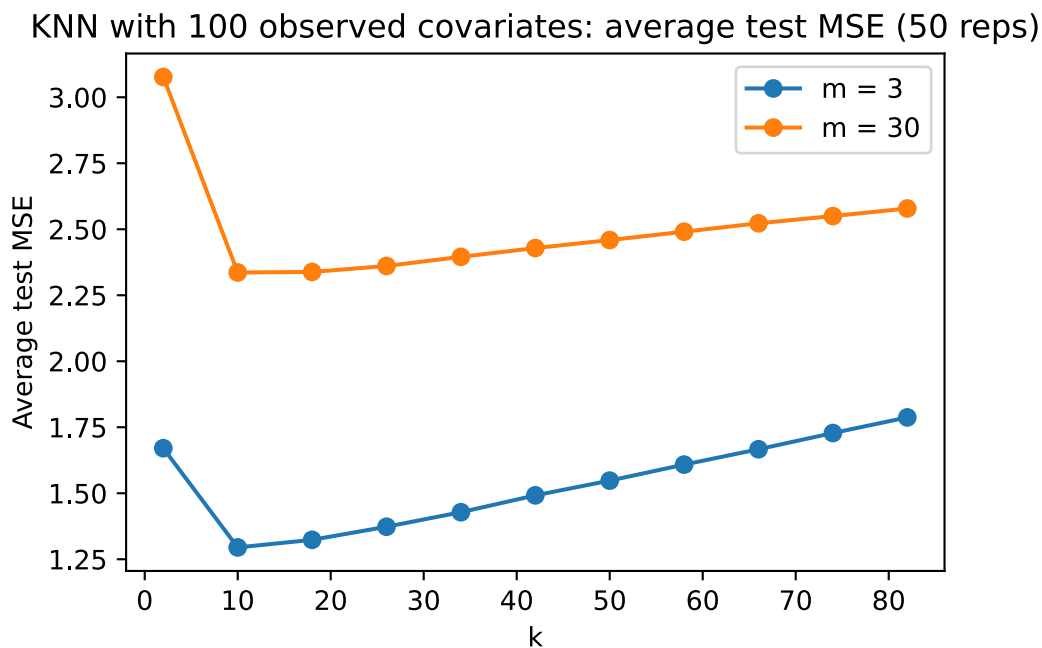


Figure 2: Average test MSE of KNN over 50 repetitions, using the 100 observed covariates.

For reference, $\text{Var}(Y) = 3$ is the MSE of predicting the mean, and the Bayes MSE is $\text{Var}(\epsilon) = 1$. With $m = 3$, the best average test MSE is 1.295 at $k = 10$, fairly close to the Bayes floor. With $m = 30$, the best is 2.336 at $k = 10$, which recovers only a small part of the explainable variance. At $k = 2$ with $m = 30$ (3.077), KNN does no better than predicting the mean. The standard errors are about 0.02–0.04, so the gap between the two curves is far larger than the simulation noise. Both curves have the usual shape: $k = 2$ is too noisy, and past $k \approx 10$ the growing neighborhoods add bias.

(c) Why performance differs

Both settings have 100 observed covariates and the same response, but what matters for KNN is the **intrinsic dimension** of $\mathbf{X}$, not the number of columns. Without noise, $\mathbf{X}_i = \mathbf{A}\mathbf{Z}_i$ lies in the m -dimensional column space of $\mathbf{A}$, and distances between observations are distances between latent vectors under the fixed metric $\mathbf{A}^T \mathbf{A}$. KNN is therefore really searching for neighbors in the m -dimensional latent space.

- **When $m = 3$:** 200 training points cover a 3-dimensional space reasonably well. Two of the three latent directions (Z_1 and Z_2) are exactly the ones the response depends on, so observations that are close in $\mathbf{X}$ are also close in $Z_1 + Z_2$.
- **When $m = 30$:** 200 points in 30 dimensions are sparse, and even the nearest neighbors are far away (the curse of dimensionality). Worse, 28 of the 30 latent directions are irrelevant to Y , yet they carry roughly equal weight in the distance. This is the Setting 1 problem from Question 2 again, now in latent coordinates. Neighbors that are “close” in $\mathbf{X}$ are often not close in (Z_1, Z_2) , so KNN predictions shrink toward the overall mean.

Role of η_i , and why the data are not exactly m -dimensional. The term η_i adds independent unit-variance noise to all 100 coordinates. Because it is isotropic and independent of $\mathbf{Z}_i$, it pushes $\mathbf{X}_i$ off the column space of $\mathbf{A}$ in every direction. With probability one, the observed data span all of $\mathbb{R}^{100}$. They form a thin “slab” around an m -dimensional subspace rather than lying on it.

- **The noise is small relative to the signal.** Each signal direction has variance on the order of $\|\mathbf{a}_j\|^2 \approx 100 \cdot \frac{4}{3} \approx 133$, compared with 1 for each noise direction. The top m principal components hold on average 80.9% of the total variance of $\mathbf{X}$ when $m = 3$ and 98.6% when $m = 30$. This ordering also shows that the $m = 30$ setting is not worse because it is noisier. In relative terms it has less noise. It is worse because its signal is 30-dimensional.
- **The noise still distorts distances.** Each pairwise squared distance gets an extra $\|\eta_i - \eta_j\|^2$ term, with mean 200 and standard deviation about 28. That term is unrelated to $\mathbf{Z}$, so it randomly reorders candidate neighbors whose latent distances are similar. This matters most when neighbors are truly close, as in the $m = 3$ case.

This is also why KNN works on the handwritten digits: the 256 pixels vary mainly along a few smooth factors such as stroke shape, slant, and thickness, so their intrinsic dimension is much lower than 256.

Question 4: Classify handwritten digits

(a) A hand-written 5NN classifier

```
train = pd.read_csv("data/knn/zip-train.csv.gz", header=None).to_numpy()[1:1000]
test = pd.read_csv("data/knn/zip-test.csv.gz", header=None).to_numpy()[1:500]
ytr4, Xtr4 = train[:, 0].astype(int), train[:, 1:]      # label column excluded
from distances
yte4, Xte4 = test[:, 0].astype(int), test[:, 1:]

def knn5_classify(X_train, y_train, X_test, distance="euclidean", k=5):
    """Distances are computed directly. Distance ties go to the smaller training
    row
    (stable sort); voting ties go to the smallest digit (argmax returns the
    first max)."""
    preds = np.empty(len(X_test), dtype=int)
    for i, x0 in enumerate(X_test):
        diff = X_train - x0
        if distance == "euclidean":
            d = np.sqrt(np.sum(diff ** 2, axis=1))
        elif distance == "manhattan":
            d = np.sum(np.abs(diff), axis=1)
        else:
            raise ValueError("distance must be 'euclidean' or 'manhattan'")
```

```

        nearest = np.argsort(d, kind="stable")[:k]
        preds[i] = int(np.argmax(np.bincount(y_train[nearest], minlength=10)))
    return preds

def confusion(pred, true):
    cm = np.zeros((10, 10), dtype=int)
    for pr, tr in zip(pred, true):
        cm[pr, tr] += 1
    return pd.DataFrame(cm, index=pd.Index(range(10), name="predicted"),
                        columns=pd.Index(range(10), name="true"))

pE = knn5_classify(Xtr4, ytr4, Xte4, "euclidean")
pM = knn5_classify(Xtr4, ytr4, Xte4, "manhattan")
wE, wM = pE != yte4, pM != yte4

```

The function computes each distance explicitly with NumPy array arithmetic. It does not call a built-in distance or KNN routine.

(b) Test error and confusion matrices

```

cmE, cmM = confusion(pE, yte4), confusion(pM, yte4)
print(f"Euclidean 5NN test error: {wE.mean():.3f} ({wE.sum()} of 500 wrong)")
display(cmE)
print(f"Manhattan 5NN test error: {wM.mean():.3f} ({wM.sum()} of 500 wrong)")
display(cmM)

```

```
Euclidean 5NN test error: 0.122 (61 of 500 wrong)
```

	true	0	1	2	3	4	5	6	7	8	9
predicted											
0		118	0	7	1	0	4	2	0	2	0
1		0	66	0	0	2	0	0	0	0	1
2		0	0	39	1	0	0	2	0	0	1
3		0	0	0	23	0	4	0	0	2	0
4		0	0	1	0	27	1	1	1	0	1
5		0	0	0	1	0	13	0	0	0	0
6		0	2	0	0	0	1	41	0	2	0

	true	0	1	2	3	4	5	6	7	8	9
predicted											
7		0	0	0	0	1	1	0	34	0	4
8		0	0	3	2	0	2	0	0	37	0
9		1	0	0	1	4	0	0	1	1	41

Manhattan 5NN test error: 0.138 (69 of 500 wrong)

	true	0	1	2	3	4	5	6	7	8	9
predicted											
0		118	0	6	1	0	5	3	0	4	0
1		0	67	2	0	2	0	1	0	0	1
2		0	0	35	0	0	0	2	0	0	0
3		0	0	1	24	0	4	0	0	2	0
4		0	0	1	1	27	1	0	1	0	1
5		0	0	0	1	0	11	0	0	0	0
6		0	1	0	0	0	2	40	0	2	0
7		0	0	2	1	1	0	0	34	0	5
8		0	0	2	0	0	3	0	0	34	0
9		1	0	1	1	4	0	0	1	2	41

```
def top_pairs(cm, n_top=5):
    M = cm.to_numpy()
    out = [(a, b, M[b, a] + M[a, b], M[b, a], M[a, b]) for a in range(10) for b
in range(a + 1, 10)]
    out = sorted(out, key=lambda t: -t[2])[:n_top]
    return pd.DataFrame(out, columns=["digit a", "digit b", "total confusions",
                                     "a predicted as b", "b predicted as a"])

class_acc = pd.DataFrame({
    "test count": np.bincount(yte4, minlength=10),
    "train count": np.bincount(ytr4, minlength=10),
    "Euclidean accuracy": [np.mean(pE[yte4 == d] == d) for d in range(10)],
    "Manhattan accuracy": [np.mean(pM[yte4 == d] == d) for d in range(10)],
}, index=pd.Index(range(10), name="digit"))
```



```
display(top_pairs(cmE).style.set_caption("Most confused pairs, Euclidean").hide(axis="index"))
display(top_pairs(cmM).style.set_caption("Most confused pairs, Manhattan").hide(axis="index"))
class_acc.round(3)
```

Table 1: Most confused pairs, Euclidean

digit a	digit b	total confusions	a predicted as b	b predicted as a
0	2	7	0	7
3	5	5	1	4
4	9	5	4	1
7	9	5	1	4
0	5	4	0	4

Table 2: Most confused pairs, Manhattan

digit a	digit b	total confusions	a predicted as b	b predicted as a
0	2	6	0	6
7	9	6	1	5
0	5	5	0	5
3	5	5	1	4
4	9	5	4	1

	test count	train count	Euclidean accuracy	Manhattan accuracy
digit				
0	119	213	0.992	0.992
1	68	120	0.971	0.985
2	50	139	0.780	0.700
3	29	70	0.793	0.828
4	34	66	0.794	0.794
5	26	47	0.500	0.423
6	46	104	0.891	0.870
7	36	75	0.944	0.944
8	44	98	0.841	0.773
9	48	68	0.854	0.854

Rows of each confusion matrix are predicted digits and columns are true digits, with all ten labels included. The Euclidean test error is 0.122 and the Manhattan test error is 0.138.

Most frequent confusions. Under both distances, the same few pairs account for most mistakes:

- **2 read as 0.** This is the single largest confusion under Euclidean distance (7 cases).
- **9 read as 7, and 4 read as 9.**
- **5 read as 3 or 0.**
- **3 and 8** confused in both directions.

These pairs share large parts of their strokes: the closed loops of 0, 2, 6, and 8, the diagonal of 4, 7, and 9, and the curved right side of 3, 5, and 8.

Digit 5 is the hardest class. Only 50% of test 5s are classified correctly with Euclidean distance. The classifier also predicts 0 too often: 134 predictions of 0 against 119 true zeros. Zero is the most common digit in the training set, so an ambiguous image is more likely to find several 0s among its five neighbors.

(c) Misclassified images and a comparison of the distances

```
wrong_idx = np.flatnonzero(wE)[:3]
fig, axes = plt.subplots(1, 3, figsize=(9, 3.4))
for ax, i in zip(axes, wrong_idx):
    ax.imshow(Xte4[i].reshape(16, 16), cmap="gray_r", vmin=-1, vmax=1)
    ax.set_title(f"Test row {i}\ntrue {yte4[i]}, predicted {pE[i]}")
    ax.axis("off")
fig.tight_layout()
plt.show()
```

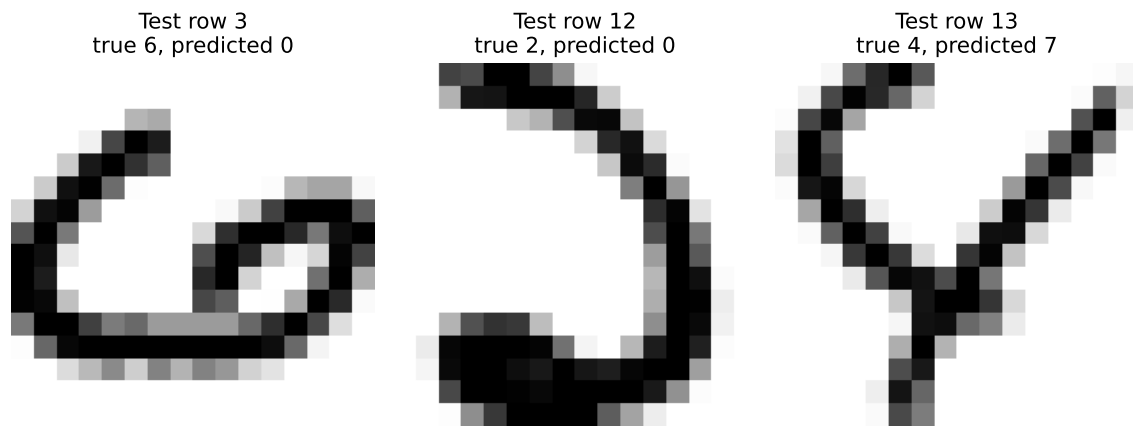


Figure 3: The first three test images misclassified by Euclidean 5NN.

Are the mistakes understandable? Mostly, yes. The figure shows the first three misclassified test images (0-based rows 3, 12, and 13).

- **Row 3 (a 6 predicted as 0).** The 6 is written with a large round bowl and a short hook, so most of its ink falls where a 0's ring would be. This mistake is easy to understand.
- **Row 12 (a 2 predicted as 0).** This is an unusual 2. It has a long curved right side and a very heavy base, with no clear diagonal stroke. A person would probably still read it as a 2, but its curved right edge and thick bottom overlap heavily with the pixels of a 0. So the error is understandable pixel by pixel, though a human reader would probably not make it.
- **Row 13 (a 4 predicted as 7).** This is an open-top 4 drawn almost like a check mark. Its long diagonal stroke from the upper right down toward the center is the defining feature of a 7. This mistake is quite understandable.

In every case the error comes from overlap of ink at the pixel level. Pixel distances do not know which strokes matter for identity; for example, whether a loop is closed or whether the vertical stroke of a 4 is present.

```
both, e_only, m_only = int(np.sum(wE & wM)), np.flatnonzero(wE & ~wM),
np.flatnonzero(~wE & wM)
disc = len(e_only) + len(m_only)
lo = min(len(e_only), len(m_only))
mcnemar_p = min(1.0, 2 * sum(math.comb(disc, j) for j in range(lo + 1)) / 2
** disc)
agree = np.mean(pE == pM)
rows_diff = np.flatnonzero(wE != wM)
pd.DataFrame({"test row": rows_diff, "true": yte4[rows_diff],
              "Euclidean pred": pE[rows_diff], "Manhattan pred": pM[rows_diff]})
```

	test row	true	Euclidean pred	Manhattan pred
0	32	5	5	0
1	63	2	2	7
2	154	8	8	9
3	233	1	6	1
4	234	2	2	1
5	241	5	5	6
6	264	8	8	0
7	269	8	8	0
8	311	5	5	3
9	338	6	6	0
10	352	2	2	1
11	380	3	8	3

	test row	true	Euclidean pred	Manhattan pred
12	437	5	0	5
13	490	2	2	7

Euclidean versus Manhattan. Euclidean distance did slightly better: 61 errors against 69.

- **The two are not identical.** They agree on 95.6% of the test images. 58 images are misclassified by both methods, 3 only by Euclidean, and 11 only by Manhattan. The table above lists these 14 images.
- **So yes, the two distances can have similar total error while getting different images wrong.** For example, row 380 (a 3) is misread as 8 by Euclidean but classified correctly by Manhattan. Rows 32, 264, 269, and 338 (a 5, two 8s, and a 6) are classified correctly by Euclidean but all called 0 by Manhattan.
- **Why the extra Manhattan mistakes might happen.** One plausible reason: squaring emphasizes a few large pixel disagreements, like ink against background. Absolute differences give relatively more weight to many small disagreements in grey level or stroke thickness, so thick-stroked images drift toward the thick-looped 0 class.
- **The overlap is large, though.** The hard images (the unusual handwriting shown above) are hard for both metrics, and they make up most of the errors. The difference in error counts rests on only 14 discordant images. An exact McNemar test gives a two-sided $p \approx 0.057$, so the evidence that Euclidean is truly better is only borderline for this sample.

Question 5: Create a plot-formatting skill

(a) The skill

The final `SKILL.md` is appended at the end of this section.

(b) Applying it to the Question 1 plot

I installed the skill and invoked it explicitly with the request “use stat432-plot-format on the Q1 plot.” The skill changed only the formatting. The data and the three plotted curves are the same as in Figure 1.

```
OKABE_IT0 = {"orange": "#E69F00", "blue": "#0072B2", "green": "#009E73"}

plt.rcParams.update({"font.size": 11, "axes.titlesize": 13})
fig, ax = plt.subplots(figsize=(7, 4.5))
ax.plot(ks1, bias ** 2, color=OKABE_IT0["orange"], marker="o", ms=5, lw=1.8,
ls="--", label="Squared bias")
ax.plot(ks1, var, color=OKABE_IT0["blue"], marker="s", ms=5, lw=1.8, ls=":",
label="Variance")
ax.plot(ks1, mse, color=OKABE_IT0["green"], marker="^", ms=5, lw=1.8, ls="--",
label="Mean squared error")
```

```

ax.set_title("Bias-variance decomposition of KNN at  $x_0$ ", loc="left", pad=22)
ax.text(0, 1.02, "n = 400 training points, 200 simulated training sets, Euclidean distance",
        transform=ax.transAxes, fontsize=10, color="0.35")
ax.set_xlabel("Number of neighbors (k)")
ax.set_ylabel("Error component")
ax.set_xticks(ks1)
ax.set_ylim(bottom=0)
ax.spines[["top", "right"]].set_visible(False)
ax.grid(axis="y", color="0.9", lw=0.8)
ax.set_axisbelow(True)
ax.legend(frameon=False, loc="upper right")
fig.tight_layout()
fig.savefig("q1_bias_variance_revised.png", dpi=200, bbox_inches="tight")
plt.show()

```

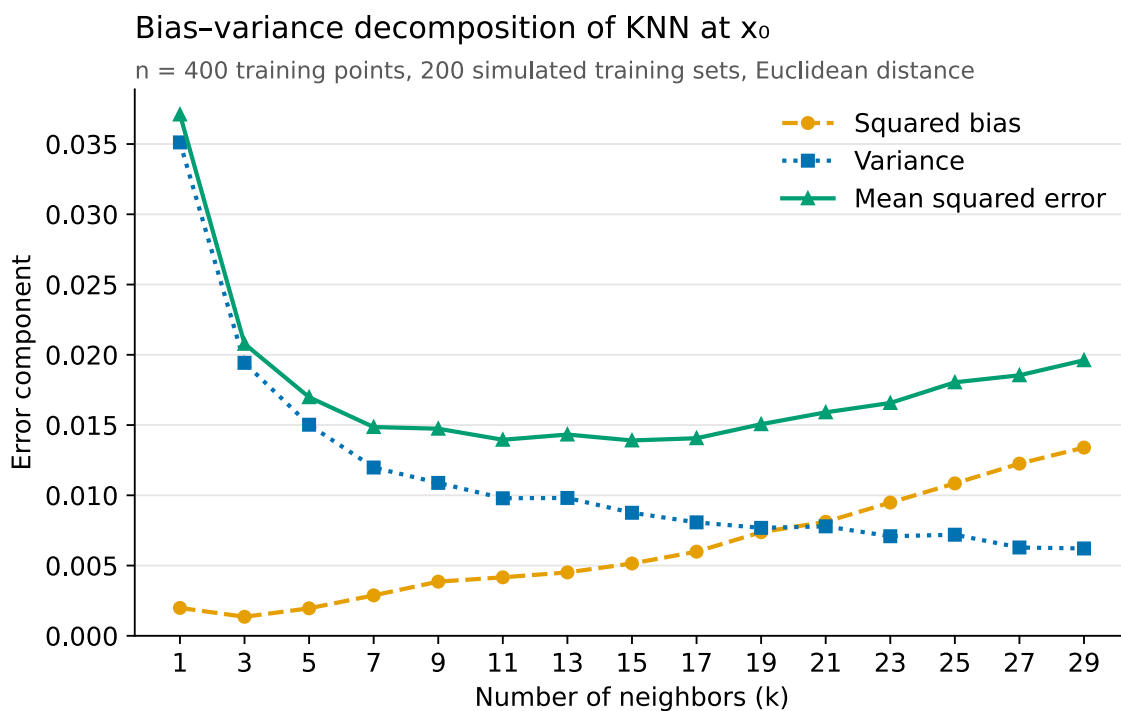


Figure 4: Question 1 plot after applying the stat432-plot-format skill.

The skill reported these changes:

- **Title and subtitle.** The title is now specific, and the setting details (n = 400, 200 training sets, Euclidean distance) moved into a subtitle.

- **Axis labels and ticks.** The axes are labeled in words (“Number of neighbors (k)” and “Error component”). There is a tick at every value of k that was used, and the y-axis starts at zero because all three quantities are nonnegative magnitudes.
- **Colors and line styles.** The default colors were replaced with the colorblind-safe Okabe–Ito palette. Each series also has its own marker (circle, square, triangle) and line style (dashed, dotted, solid), so the plot still reads when printed in grayscale.
- **Clutter and layout.** The top and right borders were removed, and only light horizontal gridlines remain. The legend now has no frame and sits in the empty upper-right area.
- **Size and export.** The figure is 7×4.5 in with 11/13 pt fonts and is saved as a 200-dpi PNG.

Comparison. The revised plot is easier to read without color. Its axis labels say what is being measured, and ticks at the actual k values make it easy to read off the minimum-MSE k and where the bias and variance curves cross. The 15 ticks sit right at the skill’s limit but still fit without crowding at this figure size. One minor cost: the subtitle adds some vertical space above the panel.

I like the changes. The revised figure is cleaner and easier to interpret than the default version, and I would keep using the skill for future homework plots.

(c) Choices and experience

Choices.

- **The description limits the skill tightly.** It applies only to STAT 432 homework plots and only on an explicit request, so it never changes plots without being asked.
- **The rules cover what the assignment asks for** (titles, axes, labels and legends, colors, sizes, margins) plus export settings. They are written as general conventions rather than tied to any homework question.
- **Colors are colorblind-safe and backed up by marker or line type**, so figures still read in grayscale printouts.
- **Two rules protect the analysis.** One tells the agent not to change the data or add conclusions, because a formatting tool should never alter the analysis. The other asks it to list its changes, which makes the before/after comparison easy.

Experience. The skill worked as intended on the first try. It was used only when I asked for it by name, it changed the formatting without touching the data or the plotted quantities, and it listed its changes, which made the before/after comparison straightforward. Because the first result met the rules I wrote, I did not see a need to revise the skill. I kept it installed and plan to keep using it.

Final SKILL.md:

```
---
name: stat432-plot-format
description: Improve the formatting of STAT 432 homework plots in R or Python.
Use ONLY when the user explicitly asks to apply this skill (e.g. "use stat432-plot-format on this plot"); never apply it automatically or to non-STAT 432 work.
---
```

STAT 432 Homework Plot Formatting

Revise an existing plot's formatting without changing the data, the computation, or what is plotted.

Rules

1. **Title**: one short, specific title stating what is plotted (e.g. "Test MSE vs. number of neighbors"). No trailing period. Put setting details (sample size, number of repetitions) in a subtitle or caption, not the title.
2. **Axes**: every axis gets a label in words, with units or the symbol in parentheses when useful (e.g. "Number of neighbors (k)"). Use tick marks at the actual values of a discrete tuning parameter when there are 15 or fewer. Start the y-axis at zero only for quantities that are naturally nonnegative and compared as magnitudes; use a log scale when values span more than two orders of magnitude, and say so in the label.
3. **Labels and legends**: use readable names, not variable names (write "Squared bias", not ``bias2``). Place the legend inside the panel where it covers no data, or directly label lines at their ends. Omit the legend for a single series.
4. **Colors**: use a colorblind-safe palette (Okabe-Ito or viridis). Keep at most 5 distinct colors per panel; distinguish series by marker shape or line type as well as color so the plot survives grayscale printing.
5. **Sizes**: default figure size 7×4.5 in (one panel). Base font 11 pt, title 13 pt. Line width about 1.5–2 pt; markers small enough not to hide each other.
6. **Margins and layout**: remove wasted whitespace (``tight_layout()`` / ``bbox_inches="tight"`` in Python; ``par(mar=...)`` or ``theme(plot.margin=...)`` in R). Remove top and right panel borders; use light, horizontal-only gridlines.
7. **Export**: save as PNG at 200 dpi (or PDF/SVG for vector output) with the same size used on screen.
8. **Report back**: list the changes made in 3–5 bullets and show the revised code. Do not add new analysis or annotations that assert conclusions.